

Aula 2 - Configuração do Ambiente e Primeira Aplicação Full-Stack

Duração: 4 horas (1h30 teórica + 2h30 prática)

Parte Teórica (1h30)

1. Revisão da Aula Anterior (15 min)

- Discussão sobre o projeto "Hello, Full-Stack!" (dúvidas e desafios).
- Conceitos-chave:
 - REST API (Spring Boot).
 - Consumo de API (Angular HttpClient).
 - CORS (Cross-Origin Resource Sharing).

2. Configuração Avançada do Ambiente (45 min)

Ferramentas Necessárias

Tecnologia	Configuração Recomendada
Angular	<code>npm install -g @angular/cli</code> , extensões VSCode (Angular Language Service, Material Icon Theme).
Java 17	JDK 17 (Amazon Corretto/OpenJDK), IDE (IntelliJ/VSCode + Extensão Java).
Spring Boot	Spring Initializr, Spring Tools Suite (opcional), Postman/Insomnia para testes de API.
MySQL	Docker (<code>docker run --name mysql -e MYSQL_ROOT_PASSWORD=senha -p 3306:3306 -d mysql</code>) ou instalação local.

Fluxo de Desenvolvimento

1. Backend: Criação de endpoints (Spring Boot).

2. Frontend: Consumo de API (Angular Services).
3. Banco de Dados: Modelagem e conexão (Spring Data JPA).

3. Estrutura de Projetos (30 min)

Spring Boot

plaintext

```
src/  
├─ main/  
│   └─ java/  
│       └─ com/example/  
│           ├── controller/  
│           ├── model/  
│           ├── repository/  
│           └─ service/  
└─ resources/  
    ├── application.properties  
    └─ static/ (opcional para frontend integrado)
```

Angular

plaintext

```
src/  
├─ app/  
│   ├── components/  
│   ├── services/  
│   ├── models/  
│   └─ app.module.ts  
├─ assets/  
└─ styles/
```

Parte Prática (2h30)

Atividade: CRUD Básico de Tarefas (To-Do List)

Objetivo: Criar uma API Spring Boot com MySQL e consumi-la via Angular.

Passo 1: Backend (Spring Boot + MySQL) – 1h

1. Crie um novo projeto Spring Boot com as dependências:
 - Spring Web, Spring Data JPA, MySQL Driver.

2. Configure o `application.properties`:

properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/todo_db
spring.datasource.username=root
spring.datasource.password=senha
spring.jpa.hibernate.ddl-auto=update
```

3. Crie a entidade `Task`:

java

```
@Entity
public class Task {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private boolean completed;
    // Getters e Setters
}
```

4. Crie o `TaskRepository`:

java

```
public interface TaskRepository extends JpaRepository<Task, Long> {}
```

5. Crie o `TaskController`:

java

```
@RestController
@RequestMapping("/api/tasks")
public class TaskController {
    @Autowired
    private TaskRepository repository;

    @GetMapping
    public List<Task> getAll() {
        return repository.findAll();
    }

    @PostMapping
    public Task create(@RequestBody Task task) {
        return repository.save(task);
    }
}
```

6. Teste no Postman:

- POST `http://localhost:8080/api/tasks` (Body: `{"title": "Estudar Angular", "completed": false}`).
- GET `http://localhost:8080/api/tasks` .

Passo 2: Frontend (Angular) – 1h30

1. Crie um serviço Angular para consumir a API:

bash

```
ng generate service task
```

typescript

```
@Injectable({ providedIn: 'root' })
export class TaskService {
  private apiUrl = 'http://localhost:8080/api/tasks';

  constructor(private http: HttpClient) {}

  getTasks() {
    return this.http.get<Task[]>(this.apiUrl);
  }

  addTask(task: Task) {
    return this.http.post<Task>(this.apiUrl, task);
  }
}
```

2. Crie um componente para listar/adicionar tarefas:

bash

```
ng generate component task-list
```

typescript

```
// task-list.component.ts
export class TaskListComponent implements OnInit {
  tasks: Task[] = [];
  newTask: Task = { title: '', completed: false };

  constructor(private taskService: TaskService) {}

  ngOnInit() {
    this.loadTasks();
  }

  loadTasks() {
```

```

        this.taskService.getTasks().subscribe(tasks => this.tasks = tasks);
    }

    addTask() {
        this.taskService.addTask(this.newTask).subscribe(() => {
            this.loadTasks();
            this.newTask = { title: '', completed: false };
        });
    }
}

```

3. Template HTML:

html

```

<div>
  <input [(ngModel)]="newTask.title" placeholder="Nova tarefa">
  <button (click)="addTask()">Adicionar</button>
  <ul>
    <li *ngFor="let task of tasks">
      {{ task.title }} - {{ task.completed ? '✓' : 'X' }}
    </li>
  </ul>
</div>

```

4. Adicione `FormsModule` em `app.module.ts` para usar `ngModel`.

5. Execute e teste:

- Backend: `mvn spring-boot:run`.
- Frontend: `ng serve`.

Tarefa para Casa

- Implementar as operações **DELETE** e **UPDATE** no CRUD.
- Estilizar a lista de tarefas com CSS (ex: Bootstrap).

Próxima Aula

- Tópico: Componentes Angular e Injeção de Dependência.
- Atividade prática: Dividir a aplicação em componentes reutilizáveis.

Resultado Esperado

- API Spring Boot funcional com persistência em MySQL.
- Frontend Angular consumindo a API e exibindo dados dinâmicos.
- Entendimento da estrutura básica de projetos full-stack.

https://miro.medium.com/v2/resize:fit:1400/1*E8t_N2Q6G7ht0Y5r6B6qZQ.png

(Fluxo de comunicação entre Angular e Spring Boot) 🚀